

Sentiment LSTM - Line by Line

Imports

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
```

Loads PyTorch core library, neural network modules, and functional operations.

Class Definition

```
1 class SentimentLSTM(nn.Module):
```

Creates a sentiment analysis model class inheriting from PyTorch's base neural network class.

Constructor

```
1     def __init__(self, vocab_size, embedding_dim=128, hidden_dim=64,
2                   num_layers=2, output_dim=3, dropout=0.5):
```

Defines initialization method with parameters:

- vocab_size: Number of unique words
- embedding_dim=128: Word vector size
- hidden_dim=64: LSTM hidden state size per direction
- num_layers=2: Number of stacked LSTM layers
- output_dim=3: Number of sentiment classes
- dropout=0.5: Dropout rate (50%)

```
1     super().__init__()
```

Initializes parent class.

```
1     self.embedding = nn.Embedding(vocab_size, embedding_dim,
                                   padding_idx=0)
```

Creates embedding layer to convert word indices to vectors. padding_idx=0 makes padding tokens always zero.

```

1         self.lstm = nn.LSTM(
2             embedding_dim,
3             hidden_dim,
4             num_layers=num_layers,
5             batch_first=True,
6             dropout=dropout if num_layers > 1 else 0,
7             bidirectional=True
8         )

```

Creates bidirectional LSTM layer:

- Input size: 128
- Hidden size: 64 per direction
- `batch_first=True`: Input shape is (batch, sequence, features)
- `dropout`: Applied between layers only if multiple layers exist
- `bidirectional=True`: Processes sequence forward and backward

```

1         self.dropout = nn.Dropout(dropout)

```

Creates dropout layer for regularization (randomly zeros 50% of values during training).

```

1         self.fc = nn.Linear(hidden_dim * 2, output_dim)

```

Creates output layer. Input size is $64 \times 2 = 128$ (bidirectional), output is 3 classes.

Forward Method

```

1         def forward(self, input_ids, attention_mask=None):

```

Defines forward pass. Takes word indices and optional attention mask.

```

1         embedded = self.dropout(self.embedding(input_ids))

```

Converts indices to embeddings then applies dropout. Shape: (batch, sequence, 128).

```

1         if attention_mask is not None:

```

Checks if attention mask is provided for variable-length sequences.

```

1         lengths = attention_mask.sum(dim=1).cpu()

```

Calculates actual sequence length for each sample by summing mask values. Moves to CPU.

```

1         packed = nn.utils.rnn.pack_padded_sequence(
2             embedded, lengths, batch_first=True, enforce_sorted=
3             False
4         )

```

The function `pack_padded_sequence` is used to handle batches of variable-length sequences in RNNs by removing padding tokens from computation. The input `embedded` has shape (B, T, D) and `lengths` contains the true lengths of each sequence. By setting `batch_first=True`, the batch dimension is kept as the first dimension of the tensor. The argument `enforce_sorted=False` allows using unsorted sequences, letting PyTorch handle the internal sorting automatically.

```
1 lstm_out, (hidden, _) = self.lstm(packed)
```

Processes packed sequences through LSTM. Returns outputs and hidden states. Cell states ignored.

```
1 else:
2     lstm_out, (hidden, _) = self.lstm(embedded)
```

Alternative path: processes unpacked sequences if no mask provided.

```
1 hidden_forward = hidden[-2, :, :]
```

Extracts final forward direction hidden state from last LSTM layer. Shape: (batch, 64).

```
1 hidden_backward = hidden[-1, :, :]
```

Extracts final backward direction hidden state from last LSTM layer. Shape: (batch, 64).

```
1 hidden_concat = torch.cat((hidden_forward, hidden_backward),
    dim=1)
```

Concatenates forward and backward states. Shape: (batch, 128).

```
1 return self.fc(self.dropout(hidden_concat))
```

Applies dropout then linear layer to produce 3 output logits per sample.

Model Instantiation

```
1 model = SentimentLSTM(
2     vocab_size=len(vocab),
3     embedding_dim=128,
4     hidden_dim=64,
5     num_layers=2,
6     output_dim=3, # 0=negatif, 1=neutre, 2=positif
7     dropout=0.5
8 )
```

Creates model instance with specified configuration. `vocab` must be predefined.

```
1 print(model)
```

Displays model architecture.

```
1 print(f"\n Modele cree avec {sum(p.numel() for p in model.parameters
    ()):,} parametres")
```

Calculates and prints total number of trainable parameters with comma formatting.

Tools Used

PyTorch Core (torch)

Main library for tensor operations and automatic differentiation.

Neural Network Module (nn)

Provides building blocks:

- `nn.Module`: Base class for all models
- `nn.Embedding`: Word embedding lookup table
- `nn.LSTM`: Long Short-Term Memory recurrent layer
- `nn.Dropout`: Regularization layer
- `nn.Linear`: Fully connected layer
- `nn.utils.rnn.pack_padded_sequence`: Efficient variable-length sequence handling

Functional Interface (F)

Provides stateless operations like activation functions and loss functions (not used in this code but commonly imported).